
CSC3060 Tutorial Week 1

Mingyi BAO

In this tutorial

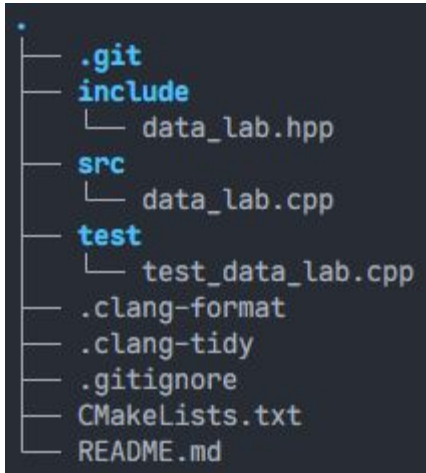
1. Essential CMake/Building Concept
2. Testing
3. C++ Fundamentals
4. Bitwise Operations



What is CMake?

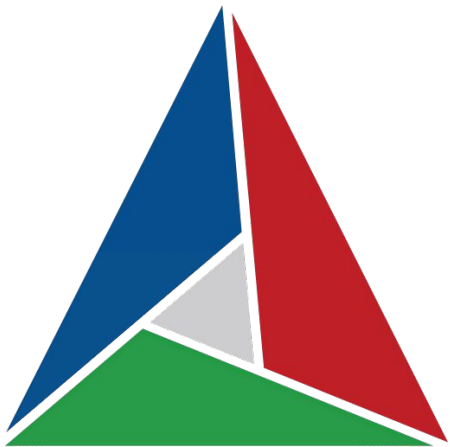
CMake is a building tool that simplifies building, testing, and packaging software by **generating native build system files** (like Makefiles or Visual Studio projects) **from simple, platform-independent configuration files.**

Project Structure



```
.
├── .git
├── include
│   └── data_lab.hpp
├── src
│   └── data_lab.cpp
├── test
│   └── test_data_lab.cpp
├── .clang-format
├── .clang-tidy
├── .gitignore
├── CMakeLists.txt
└── README.md
```

- **.git** Version control data
 - **include** Header files
 - **src** Source files
 - **test** Unit tests
 - **CMakeLists.txt** Build configuration
-



Utilize CMake

1. Write `CMakeLists.txt`
 2. Configure (Generate build files)
 3. Build/Compile (Invoke build tools to compile targets)
 4. Test
-

Write CMakeLists.txt

▲ CMakeLists.txt

```
1  cmake_minimum_required(VERSION 4.0)
2  project(data_lab LANGUAGES CXX)
3
4  set(CMAKE_CXX_STANDARD 23)
5
6  # Library target
7  add_library(data_lab src/data_lab.cpp)
8  target_include_directories(data_lab PUBLIC include)
9
10 # Test target
11 enable_testing()
12 add_executable(test_data_lab test/test_data_lab.cpp)
13 target_link_libraries(test_data_lab data_lab gtest_main)
14 add_test(NAME DataLabTests COMMAND test_data_lab)
```

Write CMakeLists.txt

`add_library (<name> <sources> )`

`add_executable (<name> <sources> )`

`target_link_libraries (<target> <item> )`

`target_include_directories ( )`

`add_test ( )`

Configure

cmake -B build

```
-- The C compiler identification is GNU 14.2.0
-- Detecting C compiler ABI info
-- Detecting C compiler ABI info - done
-- Check for working C compiler: /usr/bin/cc - skipped
-- Detecting C compile features
-- Detecting C compile features - done
-- Performing Test CMAKE_HAVE_LIBC_PTHREAD
-- Performing Test CMAKE_HAVE_LIBC_PTHREAD - Success
-- Found Threads: TRUE
-- Configuring done (1.6s)
-- Generating done (0.0s)
-- Build files have been written to: /home/bmyjacks/dev/data_lab/build
➔ data_lab
```

- Create a directory named “**build**”
 - Generate the low-level build configurations
 - Keep your root folder clean
-

Build/Compile

Run `cmake --build build`

- Invoke actual building tools to compile targets

```
[ 66%] Linking CXX executable test_data_lab
[ 66%] Built target test_data_lab
[ 75%] Building CXX object _deps/googletest-build/googlemock/CMakeFiles/gmock.dir/src/gmock-all.cc.o
[ 83%] Linking CXX static library ../../../../lib/libgmock.a
[ 83%] Built target gmock
[ 91%] Building CXX object _deps/googletest-build/googlemock/CMakeFiles/gmock_main.dir/src/gmock_main.cc.o
[100%] Linking CXX static library ../../../../lib/libgmock_main.a
[100%] Built target gmock_main
○ → data_lab █
```

Test

```
1: [=====  
1: [-----] Global test environment set-up.  
1: [-----] 3 tests from AddTest  
1: [ RUN      ] AddTest.AddPositiveNumbers1  
1: [         OK ] AddTest.AddPositiveNumbers1 (0 ms)  
1: [ RUN      ] AddTest.AddPositiveNumbers2  
1: /home/bmyjacks/dev/data_lab/test/test_data_lab.cpp:9: Failure  
1: Expected equality of these values:  
1:   data_lab::add(10, 20)  
1:     Which is: 30  
1:   3  
1:  
1: [  FAILED   ] AddTest.AddPositiveNumbers2 (0 ms)  
1: [ RUN      ] AddTest.AddPositiveNumbers3  
1: [         OK ] AddTest.AddPositiveNumbers3 (0 ms)  
1: [-----] 3 tests from AddTest (0 ms total)  
1:  
1: [-----] Global test environment tear-down  
1: [=====  
1: [-----] 3 tests from 1 test suite ran. (0 ms total)  
1: [  PASSED   ] 2 tests.  
1: [  FAILED   ] 1 test, listed below:  
1: [  FAILED   ] AddTest.AddPositiveNumbers2
```

Run **ctest --test-dir build**

- Run all unit tests
 - Show which tests failed
-

Unit test

Unit test are helpful tools to:

- Test functions in isolation
 - Verify something works correctly (as expected)
 - Narrow bugs
-

Unit test - Examples

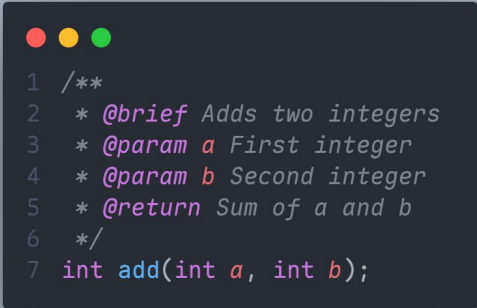


```
1 TEST(TestSuiteName, TestCaseName) {  
2     int a = 5, b = 3;  
3     EXPECT_EQ(data_lab::add(a, b), 8);  
4 }
```

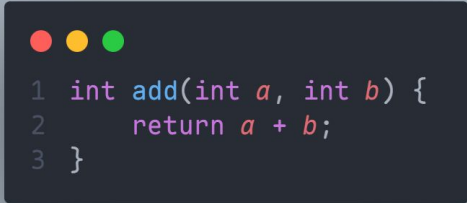
More matches: <https://google.github.io/googletest/reference/matchers.html>

C++ Fundamental

Header files vs Implementation



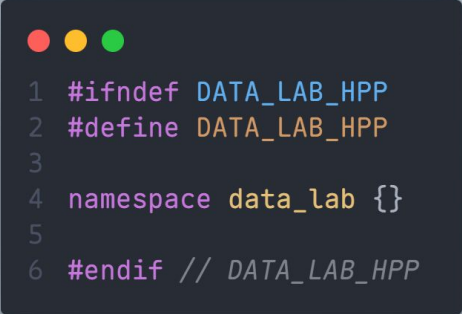
```
1 /**
2  * @brief Adds two integers
3  * @param a First integer
4  * @param b Second integer
5  * @return Sum of a and b
6  */
7 int add(int a, int b);
```



```
1 int add(int a, int b) {
2     return a + b;
3 }
```

C++ Fundamental

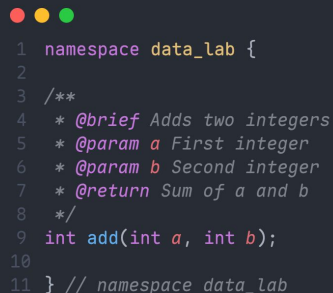
Include guards



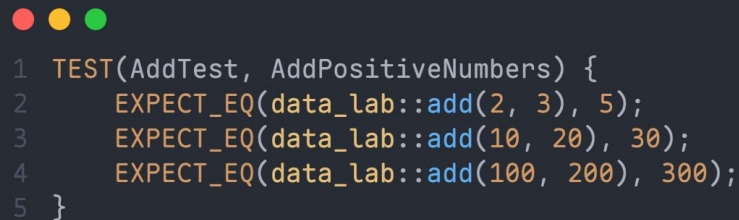
```
1 #ifndef DATA_LAB_HPP
2 #define DATA_LAB_HPP
3
4 namespace data_lab {}
5
6 #endif // DATA_LAB_HPP
```

C++ Fundamental

Namespaces



```
1 namespace data_lab {  
2  
3 /**  
4  * @brief Adds two integers  
5  * @param a First integer  
6  * @param b Second integer  
7  * @return Sum of a and b  
8  */  
9 int add(int a, int b);  
10  
11 } // namespace data_lab
```



```
1 TEST(AddTest, AddPositiveNumbers) {  
2     EXPECT_EQ(data_lab::add(2, 3), 5);  
3     EXPECT_EQ(data_lab::add(10, 20), 30);  
4     EXPECT_EQ(data_lab::add(100, 200), 300);  
5 }
```

Bitwise Operations

- AND
 - OR
 - XOR
 - NOT
 - LSHIFT
 - RSHIFT
-

Bitwise Operations Examples

	AND	OR	XOR	NOT	Left Shift	Right Shift
5 op 3 0101 op 0011	1 0001	7 0111	6 0110	~5 1010	40 0010 1000	0 0000
